Building the AI Engine

Version 1, 2026-06-09

Introduction & Overview

The Shadow AI Crisis

The CIO at your enterprise just called an emergency meeting. She projects a slide showing two deeply concerning charts.

"We have a $1.5 million 'Shadow AI' problem," she begins, pointing to the first chart. "Developers across five different departments are expensing unmanaged, external LLM API keys. We have zero visibility into what data is leaving our network or how many tokens we are consuming."

She points to the second chart. "At the same time, we've invested over $500,000 in internal GPU infrastructure. Yet, our data scientists are waiting weeks for capacity because resources are hoarded by single teams or sitting idle."

The business needs to regain control. By centralizing AI operations, enterprises have realized over $5 million in cost avoidance, achieved a 40%+ reduction in GPU hours, and delivered 2x faster model responses.

Your mandate is clear: Transform this fragmented chaos into a secure, governed, and highly scalable **Enterprise AI Factory**.

Course Structure & Prerequisites

This is a practical, hands-on series. Rather than just showing you the "happy path," we walk through the exact configurations and capabilities you will face in production. Each of the 8 modules follows a consistent three-part structure:

1. **Readiness Assessment** — Test your existing knowledge before studying the material.
2. **Runbook** — Architecture overview, anti-patterns, configuration cheat sheets, and business value context.
3. **Interactive Labs** — UI click-path walkthroughs that let you practice without dedicated lab infrastructure.

To get the most out of this course, you should understand:

- Red Hat OpenShift administration basics.
- Generic GPU enablement and Kubernetes resource requests.
- Basic API and networking principles.

Product Overview by Persona

OpenShift AI 3.4 brings massive updates that directly solve our CIO's crisis, targeting three core roles within the enterprise.

The Platform Engineer (Your Primary Role)

You are responsible for securing the infrastructure, optimizing hardware utilization, and providing visibility into resource consumption.

- **Modules 1-2:** Stand up the platform and govern access with MaaS subscriptions, token quotas, and self-service API keys.
- **Modules 3-4:** Scale massive models across GPUs with llm-d and Kueue, and secure agentic workflows with the MCP Catalog and EvalHub.
- **Modules 5-6:** Govern hardware sharing with Hardware Profiles and lock down model provenance with the Registry and OCI Modelcars.
- **Modules 7-8:** Enable rapid prototyping with Llama Stack and the Gen AI Playground, then build enterprise observability with financial showback.

The Data Scientist

Focused on discovering the right models, ensuring their accuracy, and optimizing RAG pipelines.

- **Module 4:** Use the MCP Catalog's "Tool calling" filter to identify validated agentic models, then run EvalHub benchmarks to guarantee safety.
- **Module 6:** Register, version, and promote fine-tuned models through the Model Registry with immutable OCI Modelcars.

The Application Developer

Needs frictionless, self-service access to models and tools to build intelligent applications quickly.

- **Module 2:** Generate self-service API keys scoped to MaaS subscriptions — no IT tickets required.
- **Module 7:** Prototype foundation models and test agentic MCP tools in the Gen AI Playground before writing any application code.

Your IT Support Queue

The CIO's mandate has landed as 8 open tickets in your IT support queue. Each module in this course resolves one ticket. By the end, every ticket is closed and the AI Factory is operational.

Ticket	Priority	Problem Statement	Resolution
AIOPS-001	P1	**Platform deployed but models unreachable.** A junior admin manually created a Kubernetes Route to expose a model endpoint, completely bypassing the Gateway API. MaaS token authentication, rate limiting, and Prometheus scraping are all broken.	Module 1

Ticket	Priority	Problem Statement	Resolution
AIOPS-002	P1	**$1.5M in shadow API spend.** 50 developers are sharing a single root OpenAI API key embedded directly in source code. Finance cannot attribute costs. A runaway loop in one developer's code exhausted the enterprise token limit, taking down all 50 applications.	Module 2
AIOPS-003	P1	**70B model crashes with OOM on every deploy.** The team keeps increasing Pod replicas hoping the model will "spread out" — but each replica attempts to load the entire 140GB model into a single 24GB GPU. It just copies the failure four times.	Module 3
AIOPS-004	P1	**Agent leaked Kafka credentials.** A developer embedded personal Confluent Cloud API keys directly into a LangChain script and exposed the agent to the internet. A prompt injection attack could exfiltrate sensitive shipping manifests with zero audit trail.	Module 4
AIOPS-005	P2	**Production model silently corrupted.** A data scientist uploaded experimental weights to the same S3 URI used by the production InferenceService. The next pod restart loaded the broken model — no version control, no rollback, no audit trail.	Module 5
AIOPS-006	P2	**$50K A100 GPU assigned to an idle Pandas notebook.** A data scientist selected "GPU Node" for basic exploratory analysis and left the notebook running over the weekend. Meanwhile, a critical LLM deployment failed with `Insufficient quota for nvidia.com/gpu`.	Module 6
AIOPS-007	P2	**Developers waiting 3 weeks for a RAG stack.** The platform team handed off three disconnected endpoints (LLM, embeddings, vector DB). Developers spent weeks writing fragile LangChain glue code. When any backend changed, everything broke.	Module 7
AIOPS-008	P2	**CIO demands cost attribution by Friday.** The team is parsing raw pod logs with a custom Python sidecar to count tokens for showback. It misses cached requests, overloads the logging infrastructure, and the report is inaccurate.	Module 8

Ready to Start Building?

You have 8 open tickets and one mandate: build the Enterprise AI Factory. Each module closes one ticket — by the end, your queue is clear and your platform is production-ready.

Proceed to Module 1: Platform Setup & Readiness.

1: Platform Setup & Readiness

<iframe type="text/javascript" src='_attachments/s1-quiz.html' style="width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>

```
---



---

:docname: section1
:page-module: chapter1
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: Platform Setup
:description: Theoretical architecture and component mapping for OpenShift AI 3.4
setup.
[#chapter1:section1]
=== Runbook: Platform Setup

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter1:section1:::_understanding_the_architecture_layers]
==== Understanding the Architecture Layers
Setting up OpenShift AI is not a single-click installation; it is a layered
architecture. Platform Engineers must understand the distinction between the
management plane and the activation plane.

* **The Operator (Management Plane):** Installing the Red Hat OpenShift AI Operator
from the **Ecosystem / Software Catalog** simply lays down the management plane. It
does *not* actually install the AI components (like Jupyter, MLflow, or KServe).
* **DataScienceClusterInitialization (DSCI - The Foundation):** The DSCI custom
resource handles global, cluster-wide configurations that must exist before components
spin up. This includes injecting cluster-wide trusted CA certificate bundles and
enabling the centralized OpenTelemetry/Prometheus observability stack.
* **DataScienceCluster (DSC - The Activation Layer):** The DSC custom resource defines
exactly which components are instantiated in the cluster by setting their
`managementState` to `Managed` or `Removed`.
** *Note for 3.4:* The MLflow Operator is now a natively managed component directly
within the DSC, removing the need for legacy feature flags.

[discrete#chapter1:section1:::_the_paradigm_shift_the_kubernetes_gateway_api]
==== The Paradigm Shift: The Kubernetes Gateway API
```

```
In previous versions of OpenShift AI, configuring KServe required OpenShift Service
Mesh and OpenShift Serverless (Knative).

**In OpenShift AI 3.4, the KServe Serverless deployment mode is officially
deprecated.**

Instead, Platform Engineers must use **KServe RawDeployment mode**, which leverages
the Kubernetes Gateway API for path-based routing. This modern architecture replaces
individual OpenShift Routes and sidecar-heavy Service Mesh architectures with a single
Gateway entry point (e.g., `openshift-ai-inference` Gateway). Advanced routing, token
authentication, and distributed serving (`llm-d`) are now handled natively via the
Gateway API paired with Red Hat Connectivity Link (Kuadrant).

[discrete#chapter1:section1:::_selecting_the_right_3_4_serving_runtime]
==== Selecting the Right 3.4 Serving Runtime
OpenShift AI 3.4 provides highly specialized runtimes out-of-the-box. Selecting the
wrong runtime will cause immediate Out-Of-Memory (OOM) errors or massive latency.

* **vLLM ServingRuntime:** The standard for serving Single-Node Large Language Models
on NVIDIA/AMD GPUs. It uses advanced techniques like PagedAttention and continuous
batching.
* **Distributed Inference (`llm-d`):** Used when an LLM is too large to fit into the
memory of a single GPU. It spans model weights across multiple nodes/GPUs using Ray
and the Leader Worker Set operator.
* **MLServer ServingRuntime:** Now GA in 3.4, this is the designated runtime for
classical, structured-data machine learning models built with Scikit-Learn, XGBoost,
and LightGBM.
* **Automatic Runtime Suggestion:** When deploying via the UI, OpenShift AI analyzes
the selected model framework (e.g., ONNX, SafeTensors) and matches it against the
accelerator identifier defined in the chosen Hardware Profile. This allows the system
to automatically suggest the optimal serving runtime, reducing manual configuration
errors.

[discrete#chapter1:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*

**The Scenario:** A junior admin deployed the OpenShift AI 3.4 Operator and activated
KServe in the DSC. A Data Scientist deploys a model, but external applications cannot
reach the endpoint. The junior admin decides to manually write a Kubernetes `Route`
YAML file exposing port 8080 of the model's pod to the public internet.

**Why this is an Anti-Pattern:** By bypassing the Gateway API and creating a manual
route, the admin has completely broken the platform's security and observability. The
Models-as-a-Service (MaaS) token authentication, rate limiting, and Prometheus metrics
scraping rely entirely on traffic passing through the centralized `openshift-ai-
inference` Gateway. In 3.4, external ingress should *only* be configured by checking
the "Make deployed models available through an external route" box in the UI, which
registers the route securely with the Gateway.

[discrete#chapter1:section1:::_configuration_cheat_sheet_openshift_ai_3_4]
```

```
==== Configuration Cheat Sheet: OpenShift AI 3.4
As a Platform Engineer, you will be expected to know these specific configurations to
ensure the platform operates smoothly and upgrades safely:

* **Operator Upgrades (OLM):** When installing the operator, always select the
**Manual** update approval strategy. OpenShift AI requires sequential updates.
"Manual" ensures you don't accidentally skip a required intermediate minor version
during cluster maintenance.
* **Monitoring User Workloads:** Activating the observability stack in the DSCI
monitors the *platform*. To ensure Prometheus automatically scrapes metrics from a
data scientist's custom model pod, you must apply the
`monitoring.opendatahub.io/scrape=true` label to their workload.
* **Prerequisites for `llm-d`:** If you plan to use Distributed Inference (`llm-d`),
the OpenShift AI operator alone is not enough. You must pre-install the **cert-manager
Operator**, the **Red Hat Connectivity Link Operator** (for gateway routing), and the
**Leader Worker Set Operator** (to manage the multi-node pod topology).
* **Custom Namespaces:** If your organization requires custom namespaces for
applications or workbenches instead of the defaults (e.g., `redhat-ods-applications`,
`rhods-notebooks`), these namespaces must be created and labeled *before* installing
the OpenShift AI Operator. Never rename default system namespaces.
* **Granular Workbench RBAC:** Platform Engineers can assign fine-grained RBAC roles
(`admin`, `edit`, `view`) on individual workbenches, enabling multi-tenant project
namespaces where users see only their own environments. This replaces the legacy
approach of one-namespace-per-team isolation.
* **Custom Component Lifecycle:** When deploying custom dashboard components or
integrating third-party tools, apply the `opendatahub.io/managed: true` annotation to
the resource. This signals the operator to include the resource in its lifecycle
management (upgrades, reconciliation).
* **Secure Supply Chain:** For regulated environments, OpenShift AI 3.4 supports
configuring workbench images to pull packages from the **Red Hat Python Index** – a
curated, vulnerability-scanned subset of PyPI that ensures a secure software supply
chain for data science notebooks.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter1
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: Building the Engine Room
:description: Interactive UI click-paths for installing OpenShift AI 3.4.
[#chapter1:section2]
=== Labs: Building the Engine Room
```

```
It is time to step into the Engine Room. In these interactive labs, you will execute
the "Happy Path" to install the Operator, activate the centralized observability
stack, and deploy a smoke-test model using the new Gateway API routing.

[discrete#chapter1:section2:::_lab_1_operator_installation_core_setup]
==== Lab 1: Operator Installation & Core Setup
**Goal:** Install the OpenShift AI Operator and activate the core components using the
`DataScienceCluster` resource.

.Arcade: Install and Activate
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. In the OpenShift web console, log in as a cluster administrator.
2. Navigate to **Ecosystem -> Software Catalog** and install **Red Hat OpenShift AI**.
3. Navigate to **Ecosystem -> Installed Operators** and click the Operator.
4. Click the **Data Science Cluster** tab, then click **Create DataScienceCluster**.
5. Select **YAML view**. Change the `managementState` from `Removed` to `Managed` for
the following core components: `kserve`, `kueue`, `mlflowoperator`, and
`modelregistry`.
6. Click **Create**.
====

[discrete#chapter1:section2:::_lab_2_configuring_the_serving_environment_observability
]
==== Lab 2: Configuring the Serving Environment (Observability)
**Goal:** Activate the centralized observability stack to monitor AI health and user
model metrics natively.

.Arcade: Enable Observability
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Navigate to **Operators -> Installed Operators -> Red Hat OpenShift AI**.
2. Click the **DSC Initialization** tab and open the `default-dsci` instance.
3. Select the **YAML** tab. Under `spec.monitoring`, set `managementState: Managed` to
deploy the Prometheus and OpenTelemetry stack. Click **Save**.
4. To enable the Dashboard UI, navigate to **Home -> API Explorer** and search for
`OdhDashboardConfig`.
5. Open the `odh-dashboard-config` instance YAML. Under `spec.dashboardConfig`, add
`observabilityDashboard: true` and click **Save**.
====
```

```
[discrete#chapter1:section2:::_lab_3_the_smoke_test]
==== Lab 3: The Smoke Test
**Goal:** Deploy a test model using the UI to verify KServe RawDeployment and Gateway
API routing are operational.

.Arcade: Deploy Smoke Test
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Open the **Red Hat OpenShift AI** dashboard via the Application Launcher.
2. Navigate to **Data Science Projects** and create a project named `smoke-test-
project`.
3. In the **Deployments** tab, click **Deploy model**.
4. Fill out the wizard using the **MLServer ServingRuntime** and the **Scikit-learn**
framework.
5. *Critical Step:* Under the Model route section, check the box for **"Make deployed
models available through an external route"** to trigger the Gateway API ingress.
6. Click **Deploy** and wait for the status checkmark to turn green.
====

**Module Complete!** You have successfully laid the foundation. You are now ready to
govern this platform using Models-as-a-Service in Module 2.

:!navtitle:
:!description:

:docname: index
:page-module: chapter2
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#chapter2:index]
== 2: Governing Access with Models-as-a-Service

++++
<iframe type="text/javascript" src='_attachments/s2-quiz.html' style="width: 1024px;
height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen
allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++

:docname: section1
:page-module: chapter2
:page-relative-src-path: section1.adoc
```

```
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: MaaS & Observability
:description: Theoretical architecture, cheat sheet, and Anti-Pattern for MaaS.
[#chapter2:section1]
=== Runbook: MaaS &amp;amp; Observability

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter2:section1:::_the_centralized_gateway_vs_direct_static_api_keys]
==== The Centralized Gateway vs. Direct Static API Keys
In traditional "Shadow AI" environments, developers bypass IT by directly requesting
and embedding vendor API keys (like OpenAI or AWS Bedrock) into their source code.
This is a massive vulnerability: if a key leaks, there is no way to instantly restrict
access, and the enterprise has zero visibility into which team is driving up the cloud
bill.

**Models-as-a-Service (MaaS)** solves this using the MaaS API Gateway (backed by
Kuadrant and Red Hat Connectivity Link). Instead of handing out the root provider
keys, the Platform Engineer stores the root provider key in a secure Kubernetes Secret
and registers an `ExternalModel` resource.

The MaaS Gateway acts as a proxy using a **Two-Tier Authentication** pattern:
1. Users authenticate to the Gateway using scoped, self-service MaaS API Keys.
2. The Gateway tracks their token consumption against their assigned quota, applies
rate limits, and then securely injects the root provider key into the outbound request
`[Source: Red Hat OpenShift AI Self-Managed-3.4-Govern LLM access with Models-as-a-
Service - Two-tier authentication]`.

[discrete#chapter2:section1:::_the_architecture_of_access_groups_subscriptions_and_key
s]
==== The Architecture of Access: Groups, Subscriptions, and Keys
Access in MaaS 3.4 is decoupled into distinct layers to provide maximum GitOps
flexibility:

* **OpenShift User Groups:** Defines the user's identity (e.g., `data-scientists`,
`ml-engineers`).
* **MaaS Subscriptions (`MaaSSubscription`):** Maps a User Group to specific models
and enforces strict token quotas (e.g., 500k tokens per hour). If a user belongs to
multiple groups, the system automatically selects the Subscription with the highest
numeric Priority Level (e.g., Priority 10 takes precedence over Priority 0) when
generating an API key without explicit subscription selection `[Source: Red Hat
OpenShift AI Self-Managed-3.4-Govern LLM access with Models-as-a-Service - Priority
levels]`.
* **Authorization Policies (`MaaSAuthPolicy`):** Grants the User Group the actual
network permission to pass through the API Gateway `[Source: Red Hat OpenShift AI
```

```
Self-Managed-3.4-Govern LLM access with Models-as-a-Service - Models-as-a-Service
authorization policies]`.
* **API Keys:** Developers generate their own self-service keys scoped to their
Subscription `[Source: Red Hat OpenShift AI Self-Managed-3.4-Govern LLM access with
Models-as-a-Service - Self-service API key management]`.

[discrete#chapter2:section1:::_configuration_cheat_sheet_granting_a_new_team_access]
==== Configuration Cheat Sheet: Granting a New Team Access
To provision access for a new team, a Platform Engineer completes these exact steps:

1. **Publish the Model:** Create a `MaaSModelRef` (for internal models) or an
`ExternalModel` (for external APIs like OpenAI) to register the endpoint.
2. **Create the Subscription:** Navigate to **Settings -> Subscriptions**. Assign the
OpenShift User Group, select the model, and define the Token Rate Limit (e.g.,
1,000,000 / 1h).
3. **Create the Authorization Policy:** Check the "Create matching authorization
policy" box during Subscription creation, or manually create a `MaaSAuthPolicy`
matching the Group to the Model.
4. **Inform the Developer:** Instruct developers to navigate to **Gen AI studio -> API
keys** to generate their scoped, self-service tokens.

*Prerequisites:* MaaS requires a provisioned **PostgreSQL database (version 14+)** to
manage API key lifecycles — OpenShift AI does not provision this automatically. The
database credentials must be stored in a Kubernetes Secret named `maas-db-config` in
the `models-as-a-service` namespace. Additionally, **User Workload Monitoring (UWM)**
must be enabled on the OpenShift cluster; without it, MaaS cannot collect usage
metrics and the installation will show a `Degraded` status.

[discrete#chapter2:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*

**The Scenario: The Shadow AI Key Leak**
Your company decides to experiment with OpenAI's GPT-4o. To move fast, the Lead
Platform Engineer creates a single enterprise OpenAI account, generates a root API
key, and emails it to the 50 developers on the Analytics team. The developers embed
this key directly into their application code (`Authorization: Bearer sk-openai-...`).

**Why this is an Anti-Pattern:**

* **No Showback:** Finance receives a single massive bill from OpenAI at the end of
the month, with zero ability to determine which developer or application is
responsible.
* **No Quota Enforcement:** A runaway loop in a single developer's code can exhaust
the entire company's provider-level token limit, taking down production applications
for everyone.
* **Security Nightmare:** If a developer accidentally commits the root key to a public
GitHub repo, the Platform Engineer has to revoke the key at the provider level,
instantly breaking all 50 applications simultaneously.

**The Solution:**
```

```
The Platform Engineer should use the OpenShift AI 3.4 `ExternalModel` custom resource.
The root OpenAI API key is stored securely in a Kubernetes Secret inside the cluster.
Developers never see it. Instead, developers generate their own temporary or permanent
MaaS API keys via the OpenShift AI dashboard and send requests to the centralized
OpenShift AI Gateway.

The Gateway enforces their specific Subscription token limit (preventing runaway
code), logs their exact usage for Finance's showback reports in the Observability
Dashboard, and safely proxies the request to OpenAI. If a developer's MaaS key leaks,
the Admin can simply click "Revoke" on that specific key without affecting the other
49 developers.

[discrete#chapter2:section1:::_api_key_lifecycle_governance]
==== API Key Lifecycle & Governance
MaaS API keys carry a critical security implication: group membership is captured as a
*snapshot* at the moment the key is created. If a contractor or employee is later
removed from their OIDC group, their previously generated API keys continue to
function until they expire or are manually revoked by an administrator. Platform
Engineers must build a revocation checklist into their offboarding runbook.

To enforce organizational key hygiene, administrators configure the global maximum
expiration for all generated API keys in the `Tenant` custom resource at
`spec.apiKeys.maxExpirationDays` (e.g., 90 days). This prevents developers from
creating permanent keys that become long-lived security liabilities. The `Tenant` CR
is also where Platform Engineers configure global MaaS settings including telemetry
options for showback (covered in detail in Module 8) and Gateway references.

_(For advanced multi-tenant governance patterns — preemption policies, cross-
subscription fairness, and enterprise-scale rollout playbooks — see the supplemental
rhoai3-mass2 deep-dive content.)_

[discrete#chapter2:section1:::_enabling_self_service_at_scale]
==== Enabling Self-Service at Scale
The MaaS architecture directly enables a self-service AI platform. Developers generate
their own API keys, select their own subscriptions, and prototype in the Playground —
all without filing a ticket. The Platform Engineer's role shifts from "gatekeeper" to
"guardrail designer." Finance gets showback reports. Security gets audit trails.
Developers get velocity. This is the core value proposition when presenting MaaS to
organizational leadership.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter2
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
```

```
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: MaaS & Observability
:description: Interactive UI click-paths for configuring OpenShift AI 3.4 MaaS.
[#chapter2:section2]
=== Labs: MaaS &amp;amp; Observability

In these interactive labs, you will execute the "Happy Path" to securely proxy an
external LLM, govern user access with strict quotas, and view token showback reports
for finance.

[discrete#chapter2:section2:::_lab_1_connecting_an_external_provider]
==== Lab 1: Connecting an External Provider
**Goal:** Securely register an external model (GPT-4o) using the OpenShift console so
developers don't need the root API key.

.Arcade: Configure External Egress
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Log in to the OpenShift Console as a cluster administrator.
2. Create the root credential: Open the terminal and run: `oc create secret generic
openai-secret --from-literal=api-key=<YOUR_OPENAI_KEY> -n external-models`.
3. Navigate to **Administration -> CustomResourceDefinitions**.
4. Search for `ExternalModel` and click the resource name.
5. Click the **Instances** tab, select the `external-models` project, and click
**Create ExternalModel**.
6. Configure the YAML: Set `spec.provider: openai`, `spec.endpoint: api.openai.com`,
`spec.targetModel: gpt-4o`, and `spec.credentialRef.name: openai-secret`.
7. Click **Create**. The model is now registered in the MaaS Gateway.
====

[discrete#chapter2:section2:::_lab_2_creating_the_subscription_and_auth_policy]
==== Lab 2: Creating the Subscription and Auth Policy
**Goal:** Assign the "Analytics" team a strict quota for the newly registered GPT-4o
model.

.Arcade: Define Subscription Quotas
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Open the OpenShift AI Dashboard and navigate to **Settings -> Subscriptions**.
```

```
2. Click **Create subscription**. Name it `analytics-gpt-sub`.
3. Set the Priority level to `10`.
4. Under Groups, select the OpenShift group `analytics-team`.
5. Click **Add models**, select your new `gpt-4o` external model, and click **Add
models**.
6. Click **Add token limit**. Set the limit to `500000` tokens per `1 hour`. Click
**Save**.
7. Check the box for **"Create a matching authorization policy"** to simultaneously
grant API gateway access.
8. Click **Create subscription**.
====

[discrete#chapter2:section2:::_lab_3_generating_keys_monitoring_showback]
==== Lab 3: Generating Keys & Monitoring Showback
**Goal:** Step into the shoes of a Developer generating a key, then pivot back to the
Admin to view the financial showback report.

.Arcade: Developer Self-Service and Showback
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. *(As the Developer):* Navigate to **Gen AI studio -> API keys**.
2. Click **Create API key**. Name it `dev-testing-key`.
3. Select the `analytics-gpt-sub` subscription, set expiration to 30 days, and click
**Create**. Copy the `sk-oai-...` prefix key before closing the dialog.
4. *(As the Admin):* Navigate to **Observe & monitor -> Dashboard**.
5. Click the **Usage** tab to view the MaaS observability metrics.
6. Review the **Token Consumption by User** table to see exactly how many tokens the
developer consumed against the `gpt-4o` endpoint.
7. Click **Export as CSV** to download the billing showback report for Finance.
====

**Module Complete!** You have successfully eliminated shadow AI by providing
developers self-service access while maintaining strict financial and security
controls.

:!navtitle:
:!description:

:docname: index
:page-module: chapter3
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
```

```
[#chapter3:index]
== 3: Scaling Massive Models

++++
<iframe type="text/javascript" src='_attachments/s3-quiz.html' style="width: 1024px;
height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen
allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++

:docname: section1
:page-module: chapter3
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: Distributed Inference & Kueue
:description: Theoretical architecture and Anti-Pattern for Kueue and llm-d.
[#chapter3:section1]
=== Runbook: Distributed Inference &amp;amp; Kueue

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter3:section1:::_why_massive_llms_require_distributed_inference]
==== Why Massive LLMs Require Distributed Inference
A common misconception is that standard Kubernetes autoscaling or adding more Pod
replicas will solve Out of Memory (OOM) errors for large models. It will not.

If a data science team attempts to load a 70-Billion parameter model at 16-bit
precision, the model weights alone require approximately 140GB of VRAM. If your
cluster only has standard GPUs with 24GB to 80GB of VRAM, the model cannot physically
fit on a single device.

**Distributed Inference with `llm-d`** solves this by applying Tensor Parallelism and
Pipeline Parallelism. Instead of duplicating the entire model per pod, `llm-d`
leverages the Red Hat **Leader Worker Set (LWS) Operator** (which officially replaces
Ray in 3.4 for vLLM workloads) to slice the model's layers and distribute them across
multiple GPUs and nodes. Deploying with llm-d uses the **`LLMInferenceService`**
custom resource — not the standard `InferenceService` used for single-node vLLM
deployments. The key configuration fields are `workerSpec.tensorParallelSize` (which
determines how many GPUs on a single node share the sharded model weights) and
`pipelineParallelSize` (which handles the inter-node split across multiple machines).

[discrete#chapter3:section1:::_kueue_the_gpu_traffic_cop]
==== Kueue: The GPU Traffic Cop
Without Kueue, if ten data scientists simultaneously request a 4-GPU distributed
inference deployment on an 8-GPU cluster, Kubernetes will attempt to schedule all of
them, resulting in `InsufficientQuota` crashes or deadlocks.
```

```
Kueue acts as a centralized traffic cop. It uses a **ClusterQueue** to define the
global limits of your hardware (e.g., maximum 8 GPUs total) and **LocalQueues** in
user namespaces that point to the ClusterQueue. If resources are full, Kueue
gracefully holds the new workloads in a "Pending" state in the LocalQueue until the
required GPUs are freed up.

Enforcement is strict: when a namespace is labeled with
`kueue.openshift.io/managed=true`, a **validating admission webhook** automatically
rejects any supported workload that lacks the `kueue.x-k8s.io/queue-name` label. This
guarantees no GPU workload can bypass the queue, even if a developer attempts to
create a raw Pod directly.

[discrete#chapter3:section1:::_configuration_cheat_sheet_preparing_for_distributed_mod
els]
==== Configuration Cheat Sheet: Preparing for Distributed Models
1. **Install Prerequisites:** Install the independent Red Hat build of Kueue Operator,
Node Feature Discovery, NVIDIA GPU Operator, and the Leader Worker Set Operator.
2. **Enable Kueue:** In the DataScienceCluster CR, set
`spec.components.kueue.managementState` to `Unmanaged` (as the embedded Kueue
component is deprecated in 3.4).
3. **Define Quotas:** Create a `ResourceFlavor` (defining the node types) and a
`ClusterQueue` (defining the total GPU limits).
4. **Create the Local Queue:** In the target project namespace, create a `LocalQueue`
that references the `ClusterQueue`.
5. **Create a Hardware Profile:** In the OpenShift AI Dashboard (**Settings ->
Hardware profiles**), create a profile with the required GPUs. Under "Workload
allocation strategy", select **Local queue** and input your LocalQueue name. When a
data scientist selects this profile during deployment, the system also automatically
suggests a compatible serving runtime by matching the profile's accelerator identifier
against available runtime templates.
6. **Deploy:** Data scientists select `Distributed inference with llm-d` as the
serving runtime and apply the new Hardware Profile during deployment.

_(For advanced llm-d performance tuning — EPP optimization, prefill/decode routing,
and multi-node NCCL configuration — see the supplemental rhoai3-llmd deep-dive
content.)_

[discrete#chapter3:section1:::_intelligent_autoscaling_the_workload_variant_autoscaler
_wva]
==== Intelligent Autoscaling: The Workload Variant Autoscaler (WVA)
Traditional Kubernetes HPA scales on CPU utilization — which is useless for LLMs
because inference is fundamentally GPU and memory-bound. The 3.4 **Workload Variant
Autoscaler (WVA)** uses inference-specific saturation signals to make proactive
scaling decisions:

* **Spare KV Cache Capacity (`kvSpareTrigger`):** Scales up when average spare KV
cache capacity falls below a threshold, meaning the GPU cannot store more attention
context for concurrent requests.
* **Spare Queue Capacity (`queueSpareTrigger`):** Scales up when the average spare
request queue capacity drops, meaning the model is falling behind on incoming prompts.
```

```
These signals ensure new replicas are provisioned *before* users experience 500 errors
or timeouts — not after.

[discrete#chapter3:section1:::_gateway_discovery_technology_preview]
==== Gateway Discovery (Technology Preview)
When deploying with llm-d, the OpenShift AI dashboard now provides a **Gateway
selector dropdown** that auto-discovers available Kubernetes Gateway API resources in
the cluster. This eliminates manual Gateway name entry and reduces misconfiguration
risk during distributed inference setup — Platform Engineers configure the Gateways
once, and the UI surfaces them automatically for every deployment.

[discrete#chapter3:section1:::_distributed_training_with_kubeflow_trainer]
==== Distributed Training with Kubeflow Trainer
While the primary focus of this module is inference scaling, Platform Engineers must
also govern distributed *training* workloads that compete for the same GPU resources.

* **Kubeflow Trainer v2:** OpenShift AI 3.4 includes the Kubeflow Training Operator
v2, the standard for submitting distributed training jobs (PyTorchJob, fine-tuning,
RLHF). The v1 API (`TFJob`, legacy `PyTorchJob` CRDs) is deprecated and will be
removed in a future release — teams should migrate to the v2 `TrainingRuntime` /
`ClusterTrainingRuntime` pattern.
* **JIT Checkpointing (GA):** For long-running training jobs that span hours or days,
GPU node preemption or spot instance reclamation can destroy hours of progress. JIT
(Just-In-Time) Checkpointing automatically saves model state to persistent storage at
configurable intervals, enabling training to resume from the last checkpoint after a
disruption rather than restarting from epoch 0.
* **Kueue Integration:** Kubeflow Trainer v2 jobs integrate natively with Kueue for
GPU quota management — the same `ClusterQueue` / `LocalQueue` pattern used for
inference scheduling applies to training workloads, preventing training jobs from
starving production inference deployments.

[discrete#chapter3:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*

**The Scenario: The "More Replicas" Fallacy**
A platform engineer is asked to deploy the `Llama-3.1-70B` model. They spin up a
standard vLLM Serving Runtime on a cluster where nodes only have 24GB GPUs. The
deployment instantly crashes with an Out of Memory (OOM) error. Thinking this is a
throughput capacity issue, the engineer edits the InferenceService and increases
`spec.replicas` from 1 to 4, hoping the load will spread out and the model will load.
All 4 replicas immediately crash.

**Why this is an Anti-Pattern:**
This is a fundamental misunderstanding of VRAM memory footprint vs. HTTP request
throughput. Requesting more replicas of a standard vLLM runtime does not split the
model; it instructs Kubernetes to spin up 4 identical Pods, each attempting to cram
the entire 140GB model into a single 24GB GPU. It just copies the failure four times.

**The Solution:**
```

```
The engineer must use the **Distributed Inference with `llm-d`** runtime. By
configuring `workerSpec.tensorParallelSize` (e.g., setting it to 8), the `llm-d`
runtime pairs with the Leader Worker Set operator to actually *shard* the model
weights across 8 GPUs of 24GB each. This ensures the aggregate memory can easily
accommodate the massive LLM.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter3
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: Distributed Inference
:description: Interactive UI click-paths for configuring OpenShift AI 3.4 Kueue and
llm-d.
[#chapter3:section2]
=== Labs: Distributed Inference

In these interactive labs, you will execute the "Happy Path" to link Kueue to a
Hardware Profile and deploy a massive model across multiple GPUs.

[discrete#chapter3:section2:::_lab_1_stopping_the_gpu_turf_war_kueue]
==== Lab 1: Stopping the GPU Turf War (Kueue)
**Goal:** Ensure Kueue is enabled so workloads pend gracefully instead of crashing the
cluster.

.Arcade: Enable Kueue and LocalQueues
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Log in to the OpenShift AI dashboard as an administrator.
2. Navigate to **Settings -> Cluster settings**.
3. Locate the Kueue section and ensure the UI integration is active.
4. Navigate to your OpenShift Console terminal and ensure your data science project is
labeled for Kueue enforcement: `oc label namespace <your-project>
kueue.openshift.io/managed=true`.
5. Apply a basic `LocalQueue` YAML to your namespace named `heavy-training-queue` that
points to your cluster's default `ClusterQueue`.
====
```

```
[discrete#chapter3:section2:::_lab_2_creating_a_hardware_profile]
==== Lab 2: Creating a Hardware Profile
**Goal:** Create a UI-selectable profile that automatically links distributed
workloads to the Kueue system.

.Arcade: Hardware Profiles
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. In the OpenShift AI dashboard, navigate to **Settings -> Hardware profiles**.
2. Click **Create hardware profile**. Name it `Distributed GPU Node`.
3. Click **Add resource**. Set Resource identifier to `nvidia.com/gpu` and ensure
Requests/Limits are set appropriately for a multi-GPU slice.
4. Scroll down to the **Resource allocation** section.
5. Under **Workload allocation strategy**, select **Local queue**.
6. In the Local queue field, enter `heavy-training-queue`.
7. Click **Create hardware profile**.
====

[discrete#chapter3:section2:::_lab_3_deploying_a_distributed_model]
==== Lab 3: Deploying a Distributed Model
**Goal:** Deploy a massive model using llm-d, leveraging the queue and hardware
profile.

.Arcade: Deploying llm-d
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. In the OpenShift AI dashboard, navigate to **Data Science Projects** and open your
project.
2. Click the **Deployments** tab.
3. In the Model serving platform tile, click **Deploy model**.
4. Name the deployment `massive-70b-model`.
5. Under **Hardware profile**, select your newly created `Distributed GPU Node`.
6. Under **Serving runtime**, explicitly select `Distributed inference with llm-d`.
7. Provide your S3-compatible storage connection containing the model weights.
8. Click **Deploy**. Because you assigned the hardware profile tied to Kueue, the
underlying service is automatically labeled with `kueue.x-k8s.io/queue-name: heavy-
training-queue` and will pend gracefully if GPUs are currently in use.
====

**Module Complete!** You have successfully eliminated GPU resource conflicts and
```

```
scaled inference beyond the limits of a single node.

:!navtitle:
:!description:

:docname: index
:page-module: chapter4
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#chapter4:index]
== 4: Securing Agentic Workflows

++++
<iframe type="text/javascript" src='_attachments/s4-quiz.html' style="width: 1024px;
height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen
allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++

:docname: section1
:page-module: chapter4
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: MCP & EvalHub
:description: Theoretical architecture and Anti-Pattern for MCP and EvalHub.
[#chapter4:section1]
=== Runbook: MCP &amp;amp; EvalHub

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter4:section1:::_the_shift_to_agentic_ai_and_the_security_paradigm]
==== The Shift to Agentic AI and the Security Paradigm
Historically, AI deployments were simple Chatbots: a user asked a question, and the
LLM returned text. With "Agentic AI," the paradigm has flipped. The LLM is now an
agent making decisions to actively call external tools, execute code, or query
databases to accomplish a goal.

If developers implement this by embedding raw API keys (for AWS, GitHub, databases)
into their application code to manually glue the LLM to these tools, they create
massive security vulnerabilities.

The **Model Context Protocol (MCP)** solves this by standardizing how models connect
```

```
to data sources and tools. Instead of custom integration scripts with scattered API
keys, the Platform Engineer deploys pre-built, secure MCP Servers (like the Red Hat
OpenShift, Ansible, or partner servers) via the **MCP Catalog**. The infrastructure
handles the tool execution, and the LLM merely requests the standardized tool format.

Behind the scenes, the **`mcp-gateway`** acts as a Kubernetes-native aggregation
layer. When Platform Engineers deploy multiple MCP servers (e.g., Ansible, Confluent,
GitHub), the mcp-gateway consolidates their tools behind a single unified runtime
endpoint. Developers interact with one gateway rather than managing connections to
individual servers, and the Platform Engineer gets centralized access control,
routing, and audit logging across all tool integrations.

[discrete#chapter4:section1:::_the_value_of_evalhub_securing_ciso_approval]
==== The Value of EvalHub: Securing CISO Approval
Deploying an Agentic model without empirical safety data is a guaranteed CISO blocker.
You cannot rely on "vibes-based" manual chat testing to determine if an agent will
exfiltrate data.

**EvalHub** provides an enterprise-grade orchestration layer to automate and
standardize LLM evaluation. It allows Platform Engineers to submit evaluation jobs
against industry-standard benchmarks.

 * For Agentic security, EvalHub integrates the **garak** framework to automatically
run security vulnerability scans (like OWASP LLM Top 10 and AVID taxonomy).
 * Crucially, EvalHub seamlessly integrates with **MLflow** (now natively managed in
OpenShift AI 3.4), ensuring every evaluation run is tracked, versioned, and auditable
for compliance. MLflow's **Prompt Management** capability (GA in 3.4) lets teams
version, compare, and audit prompt templates alongside model artifacts — essential for
agentic workflows where the system prompt defines tool-calling behavior.
 * Evaluation jobs are submitted as **`LMEvalJob`** custom resources, which define the
model endpoint, benchmark suite, and compute requirements. The TrustyAI Operator
watches for these CRs and orchestrates the evaluation pipeline.
 * Beyond security scanning with garak, EvalHub supports quality evaluation of RAG
pipelines using the **RAGAS** framework. RAGAS measures retrieval quality metrics
(context precision, context recall, faithfulness, answer relevancy) to quantify
whether a Llama Stack or custom RAG deployment is returning accurate, well-grounded
answers — critical for production agentic applications that make decisions based on
retrieved context.
 * *Prerequisite:* EvalHub requires an external PostgreSQL connection string stored as
a Kubernetes Secret (`db-url`) to manage evaluation jobs.

[discrete#chapter4:section1:::_configuration_cheat_sheet_secure_agentic_deployment]
==== Configuration Cheat Sheet: Secure Agentic Deployment
1. **Vetting the Model:** Go to **AI hub -> Catalog**, check the **Tool calling**
filter, and copy the validated CLI deployment arguments (e.g., `--enable-auto-tool-
choice`).
2. **Evaluating the Model:** Submit an EvalHub job specifying the model endpoint and
selecting safety benchmarks (e.g., garak OWASP Top 10). Review the automated audit
trail in the MLflow UI.
3. **Deploying the Tool:** As an admin, navigate to **AI hub -> MCP servers
(Catalog)**, find the required tool, and deploy the MCP Server directly into the
```

```
namespace via the `mcp-lifecycle-operator`.
4. **Testing Safely:** Instruct developers to open the **Gen AI Playground**, click
the **MCP** tab, securely authorize the server with a session token, and test the
agent interactively. For immediate safety testing during prototyping, developers can
enable **Prompt Guard** directly in the Playground interface to detect prompt
injection and jailbreak attempts against their agentic prompts before committing code.

[discrete#chapter4:section1:::_production_safety_beyond_playground_guardrails]
==== Production Safety: Beyond Playground Guardrails
Prompt Guard in the Playground is a developer-facing safety net for prototyping. For
production agentic deployments, organizations should evaluate the **TrustyAI
Guardrails Orchestrator** — an input sanitization pipeline that intercepts inference
requests at the platform level to detect toxicity, PII exposure, and injection attacks
*before* they consume GPU cycles. Two concrete implementations are available via the
TrustyAI Operator: **NeMo Guardrails** and the **IBM FMS Guardrails Orchestrator**,
which deploy safety models such as HAP (Hate, Abuse, Profanity) detectors to actively
block inappropriate interactions. This ensures safety enforcement operates
continuously inside the agent loop, not only during manual testing.

This layered approach — MCP Catalog vetting -> Playground Prompt Guard -> Production
Guardrails Orchestrator — provides the defense-in-depth narrative required to gain
CISO approval for production agentic deployments. _(See supplemental rhoai3-validate
content for TrustyAI Guardrails Orchestrator implementation details.)_

[discrete#chapter4:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*

**The Scenario: The "Shadow Agent" Exfiltration Risk**
A developer on the logistics team wants to build an AI agent capable of querying real-
time shipping events from a secure internal Confluent Kafka stream. To bypass the
"slow" IT provisioning process, they embed their personal Confluent Cloud API
credentials directly into a LangChain Python script running locally. They expose the
agent to the internet to test it from their phone.

**Why this is an Anti-Pattern:**
* The developer has created a "Shadow Agent." The LLM has unrestricted access to the
Kafka stream via the developer's raw API credentials.
* If the LLM is subjected to a prompt injection attack, a malicious actor could
instruct the agent to query the Kafka stream and exfiltrate sensitive shipping
manifests to an external URL.
* Because the integration is hard-coded locally, the enterprise has zero visibility,
no audit logs, and no centralized way to revoke the tool access if compromised.

**The Solution:**
In OpenShift AI 3.4, the Platform Engineer eliminates this risk by using the **MCP
Catalog**. Instead of the developer hardcoding keys, the Platform Engineer navigates
to the MCP Catalog and deploys the Red Hat technology partner Confluent Cloud MCP
server.

The API credentials are securely managed by the cluster via the `mcp-gateway`. The
```

```
developer simply opens the Gen AI Playground, navigates to the MCP tab, and checks the
box for the Confluent tool. They can prototype the agent safely within the governed
OpenShift environment without ever touching the raw API keys.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter4
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: MCP & EvalHub
:description: Interactive UI click-paths for vetting models and deploying MCP servers.
[#chapter4:section2]
=== Labs: MCP &amp;amp; EvalHub

In these interactive labs, you will execute the "Happy Path" to find a validated
model, deploy a secure integration tool, and test it as a developer.

[discrete#chapter4:section2:::_lab_1_vetting_an_agentic_model_model_catalog]
==== Lab 1: Vetting an Agentic Model (Model Catalog)
**Goal:** Ensure you select a model empirically validated for agentic workflows,
avoiding models that will hallucinate tool calls.

.Arcade: Filtering for Tool Calling
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Log in to the OpenShift AI dashboard.
2. In the left navigation menu, click **AI hub -> Catalog**.
3. In the left filter pane, locate the Task section and select the **Tool calling**
checkbox. The catalog will refresh to display only models validated for tool-use
compatibility.
4. Click on a validated model card (e.g., `Ministral-3-14B-Instruct-2512`) to open its
details page.
5. Scroll to the YAML frontmatter section on the model card and locate the tool-
calling metadata.
6. Copy the specific required CLI arguments (e.g., `--enable-auto-tool-choice`, `--
tool-call-parser`) and the chat template.
7. Click **Deploy model**.
```

```
8. In the deployment wizard, paste the copied CLI arguments into the *Additional
serving runtime arguments* text box to ensure the model deploys with its agentic
capabilities activated. Click **Deploy**.
====

[discrete#chapter4:section2:::_lab_2_deploying_a_trusted_tool_mcp_catalog]
==== Lab 2: Deploying a Trusted Tool (MCP Catalog)
**Goal:** Deploy a pre-packaged tool from the Catalog so developers can access backend
systems without handling raw API credentials.

.Arcade: Deploying an MCP Server
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps (Admin):**
1. Ensure the `mcp-lifecycle-operator` is installed on the cluster.
2. In the OpenShift AI dashboard, navigate to **AI hub -> MCP servers**.
3. Browse the catalog and click on a trusted integration (e.g., the Confluent Cloud or
GitHub MCP Server).
4. Review the tool's capabilities and required environment variables (which will be
securely stored as secrets).
5. Click **Deploy** and select the target developer namespace (`logistics-agent-
project`).
6. The `mcp-gateway` now securely hosts this tool endpoint for the namespace.
====

[discrete#chapter4:section2:::_lab_3_testing_the_agent_safely_gen_ai_playground]
==== Lab 3: Testing the Agent Safely (Gen AI Playground)
**Goal:** Act as the developer connecting to the governed tool using the UI, verifying
the agentic workflow is operational.

.Arcade: Connecting Tools in the Playground
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++

**Guided Steps (Developer):**
1. In the OpenShift AI dashboard, navigate to **Gen AI studio -> Playground**.
2. Select your newly deployed tool-capable model from the model dropdown.
3. On the right-hand panel, click the **MCP** tab.
4. You will see the MCP Server deployed by the Admin in Lab 2. Click the **Auth** icon
to authorize your session securely.
5. *Enable Safety:* In the Playground settings, toggle on **Prompt Guard** to ensure
your test inputs are scanned for prompt injections.
6. Type a prompt that requires the tool (e.g., *"Check the Kafka stream for shipping
delays"*).
```

```
7. Watch the chat interface as the LLM successfully invokes the tool, receives the
data, and formulates an answer—all without you writing a single line of integration
code.
====

**Course Complete!** You have successfully built a secure, scalable, and governed AI
Engine Room using OpenShift AI 3.4.

:!navtitle:
:!description:

:docname: index
:page-module: chapter5
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#chapter5:index]
== 5: Model Catalog, Registry, and Storage

++++
<iframe type="text/javascript" src='_attachments/s5-quiz.html' style="width: 1024px;
height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen
allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++

:docname: section1
:page-module: chapter5
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: Registry & Modelcars
:description: Theoretical architecture and Anti-Pattern for the Model Registry and OCI
Modelcars.
[#chapter5:section1]
=== Runbook: Registry &amp;amp; Modelcars

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter5:section1:::_model_catalog_vs_model_registry_governing_the_lifecycle
]
==== Model Catalog vs. Model Registry: Governing the Lifecycle
In a mature AI architecture, Platform Engineers must define clear boundaries between
discovery and governance to prevent shadow AI assets.
```

```
* **The Model Catalog (AI Hub):** This is the entry point. It provides a curated
library to discover, evaluate, and deploy third-party, pre-trained foundation models
(like Red Hat validated Granite or Llama models).
* **The Model Registry:** This is the System of Record. When your data scientists
fine-tune those catalog models using their own data, the resulting artifact is pushed
to the Model Registry. The Registry governs the metadata (training parameters,
formats, versions, and deployment events) to ensure you always know exactly which
version of a custom model is running in production. *Prerequisite:* Deploying a
production-grade Model Registry requires configuring access to an external **MySQL
(5.x/8.x)** or **PostgreSQL (16.x+)** database for metadata storage.

*(Note: Prior to promotion to the Registry, data scientists track their experimental
training runs and evaluations logically using **MLflow**, which is now deployed as a
natively managed component in 3.4).*

[discrete#chapter5:section1:::_deep_dive_storage_connection_types]
==== Deep Dive: Storage Connection Types
How a model gets from storage into GPU memory is critical to stability and speed. In
OpenShift AI 3.4, there are three primary connection patterns.

* **URIs (Fragile):** Direct HTTP/HTTPS links to a model file. *Drawback:* Extremely
fragile and insecure for production. If the external server goes down, your deployment
fails.
* **S3/Object Storage (Standard):** The traditional method. Models are stored in S3
buckets. KServe uses an init-container to download the model into an emptyDir before
the server starts. In 3.4, the Connections API requires S3 connection Secrets to carry
the annotation `opendatahub.io/connection-type-protocol: "s3"` to be correctly
identified and validated by system components. *Drawback:* S3 objects are highly
mutable. A data scientist can overwrite a `.safetensors` file in the bucket, silently
breaking the next pod restart.
* **OCI Containers / Modelcars (The Best Practice):** A modern approach where model
weights are layered into a standard Open Container Initiative (OCI) image (e.g.,
`oci://quay.io/myorg/granite-7b:v1`).

**Benefits of OCI Modelcars:**
Modelcars provide immutable, container-native distribution. OCI images are protected
by standard container registry RBAC and image pull secrets. Furthermore, Kubernetes
node-level image caching drastically reduces model startup times because nodes don't
need to repeatedly re-download 50GB of weights from S3 over the network if the image
is already pulled.

[discrete#chapter5:section1:::_configuration_cheat_sheet_registering_storing_oci_model
s]
==== Configuration Cheat Sheet: Registering & Storing OCI Models
Platform Engineers should mandate OCI distribution for production. OpenShift AI 3.4
makes this simple via the UI:

1. Navigate to **AI hub -> Models -> Registry**.
2. Click **Register model**.
3. Instead of just referencing an S3 bucket, select **Register and store** under Model
```

```
location.
4. Define the **Model origin** (the S3 bucket where the raw weights live).
5. Define the **Model destination** (e.g., `quay.io/my-org/my-model:v1`) and provide
credentials.
6. Click **Register**. An asynchronous Kubernetes Job will download the weights,
package them via `olot` (OCI Layers On Top) into a busybox ModelCar, push it to Quay,
and register the immutable `oci://` artifact in the registry.
* **SSL Troubleshooting:** If the `kserve-storage-initializer` container fails to
download model weights from S3 with an SSL verification error, set the `S3_VERIFY_SSL`
environment variable to `false` in the connection configuration, or inject the proper
custom CA bundle via the DSCI's trusted CA configuration. This is common with self-
signed certificates on internal MinIO or Ceph deployments.

[discrete#chapter5:section1:::_deploying_from_the_registry_smart_connection_handling]
==== Deploying from the Registry: Smart Connection Handling
When deploying a model directly from the Model Registry UI, the system automatically
scans the target project for existing Data Connections. If a single matching
connection is found, it is auto-selected. If no match exists, the model location data
from the registry is auto-filled to create a new connection. This eliminates the need
for developers to manually configure S3 secrets during deployment — another step
toward true self-service model consumption.

_(For advanced storage anti-patterns — network saturation from large model pulls,
egress cost optimization, and OCI caching strategies — see the supplemental rhoai3-
storage deep-dive content.)_

[discrete#chapter5:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*

**The Scenario: The Mutated Weights Crisis**
A data scientist on the risk team deploys a critical fraud-detection model. To save
time, instead of using a formal registry, they deploy the InferenceService by pasting
a direct, public URI (`https://external-storage.com/models/fraud-detection-v1.onnx`)
into the deployment configuration.

A month later, the inference endpoint starts producing wildly inaccurate garbage
outputs. The Platform Engineer discovers that the data scientist uploaded an
experimental, broken model to that exact same external URL, silently overwriting the
original file. Because KServe pulled the new file upon pod restart, production broke
with zero audit trail.

**The Solution:**
Loading models from loose, mutable URIs or personal S3 buckets without version control
is an operational Anti-Pattern. There is no immutability, no provenance, and no way to
instantly roll back to a known-good state.

The Platform Engineer must solve this natively in 3.4 by enforcing the **Model
Registry** paired with **OCI Modelcars**. Instead of direct URIs, the data scientist
registers their approved model. The Platform Engineer uses the *Register and store*
workflow to transform the S3 weights into an immutable OCI Modelcar image pushed to
```

```
the enterprise container registry. If a new version is created, it gets a new tag
(`:v2`), ensuring the production `:v1` tag can never be silently overwritten.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter5
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: Registry & Modelcars
:description: Interactive UI click-paths for configuring the Model Registry and OCI
Modelcars.
[#chapter5:section2]
=== Labs: Registry &amp;amp; Modelcars

In these interactive labs, you will execute the "Happy Path" to activate the Model
Registry component, transform a mutable S3 model into an immutable OCI Modelcar, and
deploy it securely.

[discrete#chapter5:section2:::_lab_1_activating_the_model_registry]
==== Lab 1: Activating the Model Registry
**Goal:** As a cluster administrator, enable the Model Registry component in the
DataScienceCluster CR so data scientists can govern their assets.

.Arcade: Enable Model Registry
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Log in to the OpenShift web console as a cluster administrator.
2. Navigate to **Operators -> Installed Operators**.
3. Click **Red Hat OpenShift AI**, then click the **Data Science Cluster** tab.
4. Click `default-dsc`, and select the **YAML** tab.
5. Locate the `spec.components` section and add/update the `modelregistry` block to
`managementState: Managed` and `registriesNamespace: rhoai-model-registries`.
6. Click **Save**. Verify the `rhoai-model-registries` project is created.
====

[discrete#chapter5:section2:::_lab_2_transforming_a_model_to_an_oci_modelcar]
==== Lab 2: Transforming a Model to an OCI ModelCar
```

```
**Goal:** Take a raw S3 model and run a transfer job from the UI to package it as an
immutable OCI container.

.Arcade: Register and Store (OCI)
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. In the OpenShift AI dashboard, click **AI hub -> Models -> Registry**.
2. Select your Model Registry instance and click **Register model**.
3. In the Model location and storage section, select **Register and store**.
4. In the Model transfer job name field, enter `Package Fraud Model to OCI`.
5. In the Model origin location section, select **Object storage** and enter the raw
S3 bucket credentials where your `.onnx` files live.
6. In the Model destination location section, enter the OCI registry details (e.g.,
`quay.io/my-org/fraud-model:v1`) and your Quay robot credentials.
7. Click **Register**. You can click the notification link to monitor the Kubernetes
transfer job in real-time.
====

[discrete#chapter5:section2:::_lab_3_deploying_the_immutable_model]
==== Lab 3: Deploying the Immutable Model
**Goal:** Deploy the newly packaged OCI model directly from the UI without manual
YAML.

.Arcade: Deploy from Registry
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Navigate back to **AI hub -> Models -> Registry**.
2. Click the name of your newly registered model.
3. Click the **Versions** tab.
4. Click the action menu (**□**) beside the `v1` version and click **Deploy**.
5. In the deployment dialog, select your target Data Science Project. The system will
automatically detect or create the matching OCI Connection Secret using the
`opendatahub.io/connection-type-protocol: "oci"` annotation.
6. Select the Serving runtime and hardware profile, then click **Deploy model**.
====

**Module Complete!** You have successfully governed your AI assets by transforming
fragile, mutable files into immutable, production-grade container artifacts.

:!navtitle:
:!description:
```

```
:docname: index
:page-module: chapter6
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#chapter6:index]
== 6: Hardware Profiles &amp;amp; GPU Performance

++++
<iframe type="text/javascript" src='_attachments/s6-quiz.html' style="width: 1024px;
height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen
allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++

:docname: section1
:page-module: chapter6
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: Hardware & Performance
:description: Theoretical architecture for Hardware Profiles, MIG vs. Time-slicing,
and DCGM Observability.
[#chapter6:section1]
=== Runbook: Hardware &amp;amp; Performance

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter6:section1:::_the_hierarchy_of_hardware_governance]
==== The Hierarchy of Hardware Governance
In OpenShift AI 3.4, hardware is governed in a strict, layered hierarchy to keep
physical drivers abstracted away from end users.

* **The GPU Operator:** Whether NVIDIA, AMD, or Intel Gaudi, the lowest level is the
Operator. It manages the physical hardware driver, device plugins, and node labeling.
* **Hardware Profiles:** *(Note: Legacy "Accelerator Profiles" and "Container Sizes"
were deprecated starting with OpenShift AI 3.0 and are fully replaced by unified
Hardware Profiles in 3.4).* The modern governance layer is the Hardware Profile. This
is a custom resource (CR) that packages everything the end-user needs into a single UI
dropdown. A Hardware Profile defines the specific hardware identifiers, exact
CPU/Memory limits, and critically, the **Node Selectors and Tolerations** required to
bypass taints and route the workload to the correct node. When a data scientist
selects a Hardware Profile during deployment, the system matches the profile's
```

```
accelerator identifier against available runtime templates to automatically suggest a
compatible serving runtime.

[discrete#chapter6:section1:::_gpu_resource_sharing_mig_vs_time_slicing]
==== GPU Resource Sharing: MIG vs. Time-slicing
A Platform Engineer must choose how to divide expensive GPUs based on the workload:

* **Multi-Instance GPU (MIG):** Physically partitions the hardware. MIG provides
strict memory and fault isolation. *Use case:* Multi-tenant production model serving
where latency and security boundaries are critical.
* **GPU Time-slicing:** Uses software to divide processing time into short,
alternating slots. All workloads share the same GPU memory without strict isolation.
*Use case:* Lightweight exploratory Jupyter notebooks and data preprocessing where
full GPU isolation is unnecessary and maximizing concurrent users is the goal.

[discrete#chapter6:section1:::_multi_vendor_gpu_support]
==== Multi-Vendor GPU Support
OpenShift AI 3.4 is not exclusively tied to NVIDIA hardware. When expanding to
alternative accelerators, Platform Engineers must use the correct resource labels and
compatible software stacks:

* **AMD GPUs:** Use the `amd.com/gpu` resource label (not `nvidia.com/gpu`) in
Hardware Profiles. Deploy workbench images built with the **ROCm** platform libraries
(e.g., ROCm-PyTorch) rather than CUDA images, as ROCm and CUDA are incompatible GPU
programming frameworks.
* **Intel Gaudi Accelerators:** Use the `habana.ai/gaudi` resource label and deploy
the dedicated Intel Gaudi operator stack.

Hardware Profiles abstract this vendor complexity from end-users — a data scientist
simply selects "AMD GPU Profile" from the dropdown without needing to understand the
underlying driver differences.

_(For detailed MIG partition strategies and advanced Kueue preemption policies for
multi-team GPU fair-sharing, see the supplemental rhoai3-gpu-aas and rhoai3-hwprofiles
deep-dive content.)_

[discrete#chapter6:section1:::_the_observability_flow_dcgm]
==== The Observability Flow (DCGM)
To prevent flying blind, the infrastructure team relies on the Data Center GPU Manager
(DCGM).
1. The Platform Engineer enables the `dcgmExporter` inside the NVIDIA ClusterPolicy.
2. The exporter exposes hardware metrics (e.g., `DCGM_FI_DEV_GPU_UTIL` for
utilization, `DCGM_FI_DEV_GPU_TEMP` for temperature).
3. The centralized OpenShift AI Prometheus stack scrapes these endpoints.
4. Administrators visualize them in Grafana or the OpenShift dashboard to identify
idle GPUs or CPU-GPU bottlenecks (e.g., high CPU throttling with low GPU utilization).

[discrete#chapter6:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*
```

```
**The Scenario: The Weekend GPU Hoarder**
A data scientist needs to do some basic exploratory data analysis using Pandas and
simple scikit-learn models in a Jupyter notebook. In the OpenShift AI deployment
wizard, they simply select "GPU Node" from the dropdown. The cluster provisions their
pod and hands them a full, dedicated 80GB NVIDIA A100 GPU. They leave the notebook
running over the weekend while they are away. Meanwhile, a critical LLM deployment
fails because it cannot schedule due to `Insufficient quota for nvidia.com/gpu`.

**Why this is an Anti-Pattern:**
Assigning a full, dedicated premium GPU to a lightweight exploratory environment
wastes thousands of dollars of compute capacity. Notebooks are highly interactive and
often sit idle while users type or read code.

**The Solution:**
* **Configure GPU Time-Slicing:** The engineer modifies the NVIDIA GPU Operator by
creating a `time-slicing-config` ConfigMap with `replicas: 10`. This allows up to 10
lightweight notebook pods to share the memory and compute of a single physical GPU
concurrently.
* **Create Distinct Hardware Profiles:** The engineer creates a Hardware Profile named
"Shared Exploratory GPU" configured with a limit of `nvidia.com/gpu: 1` (which now
represents 1 slice).
* **Apply Taints and Tolerations:** To ensure production models get dedicated
hardware, the engineer taints the A100 nodes. They create a "Production A100 GPU"
Hardware Profile that contains the specific Tolerations to bypass that taint, ensuring
the A100s are reserved strictly for InferenceServices.

This pattern directly enables a self-service GPU platform: exploratory users get
immediate access to shared slices without filing tickets, while production workloads
get guaranteed dedicated hardware. The Platform Engineer designs the profiles once;
hundreds of users consume them through the UI.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter6
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: Hardware Governance
:description: Interactive UI click-paths for configuring time-slicing, hardware
profiles, and observability.
[#chapter6:section2]
=== Labs: Hardware Governance
```

```
In these interactive labs, you will execute the "Happy Path" to configure time-
slicing, enforce scheduling via Hardware Profiles, and verify utilization using
OpenShift AI metrics.

[discrete#chapter6:section2:::_lab_1_configuring_gpu_time_slicing]
==== Lab 1: Configuring GPU Time-Slicing
**Goal:** Configure the NVIDIA GPU Operator to allow multiple workloads to share a
single GPU.

.Arcade: Configure Time-Slicing
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Log in to the OpenShift web console as a cluster administrator.
2. Navigate to **Workloads -> ConfigMaps**.
3. Select the `nvidia-gpu-operator` project.
4. Click **Create ConfigMap** and select **YAML view**.
5. Enter the YAML to define the slice replicas (e.g., `name: time-slicing-config`,
`replicas: 4` for your specific GPU architecture like `tesla-t4`). Click **Create**.
6. Navigate to **Ecosystem -> Installed Operators -> NVIDIA GPU Operator ->
ClusterPolicy**.
7. Edit the `gpu-cluster-policy` YAML to reference the new ConfigMap:
`devicePlugin.config.name: time-slicing-config`. Click **Save**.
====

[discrete#chapter6:section2:::_lab_2_creating_an_enforced_hardware_profile]
==== Lab 2: Creating an Enforced Hardware Profile
**Goal:** Create a UI-selectable Hardware Profile that enforces Tolerations so
workloads schedule correctly.

.Arcade: Create Hardware Profile
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. From the OpenShift AI dashboard, click **Settings -> Hardware profiles**.
2. Click **Create hardware profile**. Name it `Production Inference Node`.
3. Click **Add resource**. Set Resource label to `nvidia.com/gpu`, and set Requests
and Limits to `1`.
4. Scroll down to the **Node selectors and tolerations** section.
5. Click **Add toleration**.
6. Set the Operator to `Exists`, Effect to `NoSchedule`, and Key to `nvidia.com/gpu`.
7. Click **Create hardware profile**.
====
```

```
[discrete#chapter6:section2:::_lab_3_identifying_gpu_underutilization_observability]
==== Lab 3: Identifying GPU Underutilization (Observability)
**Goal:** Query the DCGM metrics to identify GPUs that are allocated but sitting idle.

.Arcade: View DCGM Metrics
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. In the OpenShift web console, navigate to **Observe -> Metrics**.
2. Ensure you have the `monitoring-rules-view` role or cluster-admin rights.
3. In the query field, enter `DCGM_FI_DEV_GPU_UTIL` and click **Run queries** to view
the percentage of time the GPUs are actively processing tasks.
4. In a new query field, enter `container_cpu_cfs_throttled_seconds_total` to check
for CPU throttling. If CPU throttling is High but GPU utilization is Low, you have
identified a severe resource allocation inefficiency.
====

**Module Complete!** You have successfully governed your expensive hardware, ensuring
maximum concurrency for exploration and guaranteed quality of service for production.

:!navtitle:
:!description:

:docname: index
:page-module: chapter7
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#chapter7:index]
== 7: Enabling Rapid Prototyping (Llama Stack, Gen AI Studio, &amp;amp; AI Hub)

++++
<iframe type="text/javascript" src='_attachments/s7-quiz.html' style="width: 1024px;
height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen
allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++

:docname: section1
:page-module: chapter7
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
```

```
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Runbook: Llama Stack & Studio
:description: Core architectural concepts behind Llama Stack Distributions, Gen AI
Studio Playgrounds, and AI Hub asset utilization.
[#chapter7:section1]
=== Runbook: Llama Stack &amp;amp; Studio

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter7:section1:::_llama_stack_vs_standalone_vllm]
==== Llama Stack vs. Standalone vLLM
A standalone vLLM deployment only provides a raw inference endpoint. If developers
want to build a Retrieval-Augmented Generation (RAG) application, they are forced to
write complex "glue code" using external frameworks like LangChain. This forces
developers to manually handle document chunking, generate embeddings, orchestrate
similarity searches against vector stores, and format context windows.

Deploying a `LlamaStackDistribution` abstracts this complexity entirely. Llama Stack
unifies the inference engine, embedding models, and vector storage behind a single
OpenAI-compatible API layer. For lightweight development and experimentation, Llama
Stack supports **inline FAISS** — an in-process, in-memory vector store that runs
inside the Llama Stack server container itself with no external database required. For
production, Platform Engineers should deploy a dedicated remote vector database like
Milvus or PostgreSQL with pgvector. Using the Llama Stack `/v1/responses` API,
developers pass their queries using the built-in `file_search` tool, allowing the
platform engine to handle RAG data-retrieval routing natively.

[discrete#chapter7:section1:::_the_discovery_validation_stack]
==== The Discovery & Validation Stack
OpenShift AI 3.4 organizes rapid prototyping into two distinct administrative zones:

* **AI Hub (The Library):** The centralized Model Catalog. It eliminates deployment
guesswork by housing curated, validated foundation models. It provides infrastructure
teams with performance insights, recommended baseline arguments, and verified tool-
calling snippets directly on the model reference card.
* **Gen AI Studio & Playground (The Proving Ground):** A secure, code-free UI
interface. Instead of spinning up test scripts, developers use the interactive
Playground to swap system prompts, adjust core sampling parameters, upload raw PDFs to
test out active RAG routing, and authorize Model Context Protocol (MCP) servers to
test live Agentic tooling integrations.

[discrete#chapter7:section1:::_configuration_cheat_sheet_llama_stack_with_remote_milvu
s]
==== Configuration Cheat Sheet: Llama Stack with Remote Milvus
To stand up an enterprise-grade prototyping environment, a Platform Engineer
orchestrates the following dependencies:

1. **Deploy Metadata Database:** Provision a PostgreSQL instance (required for Llama
```

```
Stack configuration state and file reference storage).
2. **Deploy Vector Store:** Instantiate your target vector database service (e.g., a
standalone Milvus service exposing gRPC traffic on port `19530`).
3. **Deploy Core Models:** Run your target foundation LLM via vLLM and deploy an
explicit text embedding model (e.g., `nomic-embed-text`).
4. **Assemble Credentials:** Collect endpoints and access tokens into a localized
Kubernetes `Secret`.
5. **Instantiate Distribution:** Apply the `LlamaStackDistribution` Custom Resource
mapping backend engine variables directly to the corresponding Secret targets.

[discrete#chapter7:section1:::_developer_preview_exposing_vector_databases_to_the_play
ground]
==== Developer Preview: Exposing Vector Databases to the Playground
As a Developer Preview feature in 3.4, Platform Engineers can expose pre-existing,
read-only vector databases directly to the Gen AI Playground. This is achieved by
declaring vector stores through **ConfigMaps** containing connection details,
collection names, and optional metadata. These declared stores automatically appear
under **RAG / Knowledge Sources** in the Playground interface, enabling developers to
route queries through Llama Stack RAG primitives without performing manual data
ingestion or writing custom code.

This pattern is significant for organizational enablement: Platform Engineers
provision the vector infrastructure once, declare it via ConfigMaps, and every
developer on the platform can immediately prototype RAG applications through the
Playground UI — no LangChain code, no infrastructure tickets, no custom wiring.

[discrete#chapter7:section1:::_autorag_technology_preview]
==== AutoRAG (Technology Preview)
For teams scaling beyond manual RAG configuration, OpenShift AI 3.4 introduces
**AutoRAG** — an automated pipeline that evaluates multiple retrieval strategies
(chunking sizes, embedding models, reranking approaches) against a ground-truth
dataset and recommends the optimal RAG configuration. This reduces the trial-and-error
cycle of RAG tuning from weeks to hours, enabling Platform Engineers to offer
production-ready RAG endpoints as a self-service capability rather than a bespoke
engineering project.

[discrete#chapter7:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*

**The Scenario: The "Shadow Wiring" Anti-Pattern**
An engineering team is building a internal compliance RAG assistant. To support them,
the platform administrator hands off three disconnected infrastructure endpoints: an
LLM endpoint URL, an embedding model routing path, and a raw connection string to an
unmanaged external database container. The developers proceed to spend weeks writing
custom LangChain code to manage text splitting, generate vectors, execute proximity
searches, and maintain state logic. When the database is updated or a new version of
the LLM is launched, the code breaks completely, and the infrastructure team has zero
visibility into the actual retrieval requests.

**Why this is an Anti-Pattern:**
```

```
Fragmenting the underlying component layers forces the application stack to carry
heavy, fragile data orchestration code. This creates maintenance bottlenecks and
blocks centralized platform audit monitoring.

**The Solution:**
The Platform Engineer unifies the layers by deploying a declarative
`LlamaStackDistribution` Custom Resource. Llama Stack packages the inference routes,
embedding generation, and vector index bindings into a unified engine room. Developers
completely delete their fragile LangChain orchestration blocks and issue standard
calls directly to the Llama Stack OpenAI-aligned endpoints, letting the platform
handle data retrieval natively.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter7
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: Prototyping & Studio
:description: Hands-on walk-throughs for finding validated models, uploading RAG
datasets, and authorizing agentic MCP servers.
[#chapter7:section2]
=== Labs: Prototyping &amp;amp; Studio

In these interactive labs, you will trace the prototyping lifecycle: discovering
optimized configurations within the AI Hub, utilizing the Gen AI Playground to
validate a RAG document search flow, and authorizing an active tool calling server via
the Model Context Protocol.

[discrete#chapter7:section2:::_lab_1_discovering_ai_assets_in_the_hub]
==== Lab 1: Discovering AI Assets in the Hub
**Goal:** Navigate the centralized Model Catalog to evaluate validated foundational
assets and capture optimized baseline deployment settings.

.Arcade: Explore AI Hub Catalog
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. Log in to the OpenShift AI dashboard using your engineer credentials.
```

```
2. In the primary left navigation menu, expand **AI hub** and select **Catalog**.
3. Filter the curated view by clicking **Red Hat AI models** on the main menu bar to
isolate supported foundation distributions.
4. Locate and click on the **granite-3.1-8b-instruct** model card.
5. Select the **Performance Insights** tab to review validated benchmark data across
varying underlying hardware profiles.
6. Return to the main **Readme** view, locate the **Recommended Configurations** YAML
configuration block, and copy the explicitly recommended CLI tool flags (e.g., `--
enable-auto-tool-choice`).
====

[discrete#chapter7:section2:::_lab_2_testing_rag_in_the_playground]
==== Lab 2: Testing RAG in the Playground
**Goal:** Verify a model's operational context retrieval and documentation citing
capabilities within a safe, code-free interface.

.Arcade: Validate Document RAG Flows
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. From the left dashboard navigation tree, expand **Gen AI studio** and select
**Playground**.
2. Locate the central dropdown selector in the main chat header panel and choose your
deployed foundation model.
3. In the left-hand configuration inspector block, select the **Knowledge**
customization tab.
4. Click **Upload files**, browse your local path, and submit a sample reference
dataset PDF (ensure size is under `10MB`).
5. Toggle into the adjacent **Prompt** configuration panel and insert system baseline
rules instructing the model to leverage its external indexing functions (e.g., `You
MUST use the knowledge_search tool to obtain updated info.`).
6. Enter an explicit query into the main user chat field that targets information
contained only within the file. Verify that the model accurately pulls from the
context window and provides a proper reference citation.
====

[discrete#chapter7:section2:::_lab_3_enabling_and_testing_mcp_tools]
==== Lab 3: Enabling and Testing MCP Tools
**Goal:** Authorize and validate a live external tool interface using a pre-staged
Model Context Protocol (MCP) server configuration.

.Arcade: Validate MCP Tooling
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++
```

```
**Guided Steps:**
1. Open the **Gen AI studio -> Playground** workspace panel.
2. Ensure you have loaded a modern model variant possessing active, verified tool-calling capabilities.
3. In the left-hand parameters view panel, click down into the dedicated **MCP** tool tab.
4. Review the structural list of configured platform servers, locate your target connector instance (e.g., `GitHub` or `Confluent`), and click the **Auth** padlock icon to secure your active tool session with an integration token.
5. In the primary chat console input line, prompt the model to perform an environment action utilizing that connection (e.g., `Query the latest pull requests from my repository`).
6. Observe the backend execution to verify that the model successfully translates the text prompt into a structured MCP tool call, queries the external data point, and structures the response cleanly back to the chat window.
====

**Course Complete!** You have constructed the AI Engine Room from the ground up—architecting operator fabrics, scaling high-density inference pipelines, governing accelerator hardware, and standardizing rapid RAG prototyping surfaces for your enterprise development teams.

:!navtitle:
:!description:

:docname: index
:page-module: chapter8
:page-relative-src-path: index.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#chapter8:index]
== 8: Observability, Monitoring, &amp;amp; Platform Showback

++++
<iframe type="text/javascript" src='_attachments/s8-quiz.html' style="width: 1024px; height: 800px" allowfullscreen webkitallowfullscreen mozAllowFullScreen allow="autoplay *; fullscreen *; encrypted-media *" frameborder="0"></iframe>
++++

:docname: section1
:page-module: chapter8
:page-relative-src-path: section1.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
```

```
:navtitle: Runbook: Telemetry & Showback
:description: Theoretical architecture for the OpenShift AI Telemetry flow, vLLM
metrics, and Financial Showback.
[#chapter8:section1]
=== Runbook: Telemetry &amp;amp; Showback

If you passed the Readiness Assessment, use this page as your quick-reference job aid.
If you are building your foundational skills, read this carefully before proceeding to
the interactive labs.

[discrete#chapter8:section1:::_the_full_stack_telemetry_flow]
==== The Full Stack Telemetry Flow
Enterprise observability is not just about checking if a pod is running; it is about
tracking the entire AI pipeline.
In OpenShift AI 3.4, the telemetry flow operates across distinct layers:

1. **Hardware Telemetry:** Begins at the hardware level with the Data Center GPU
Manager (DCGM) exposing raw accelerator metrics (like `DCGM_FI_DEV_GPU_UTIL`).
2. **Application Telemetry:** The runtime engines (like vLLM) emit application-
specific metrics (like `vllm:time_to_first_token`).
3. **Ingestion Layer:** These metrics are scraped by **User Workload Monitoring
(UWM)**—which must be explicitly enabled on the cluster. For custom model pods
deployed by data scientists, the `monitoring.opendatahub.io/scrape: 'true'` label must
be applied to the pod template to enable Prometheus scraping. The data flows into the
**OpenTelemetry Collector (OTC)**, which acts as the standardized ingestion layer.
4. **Storage & Visualization:** From OTC, the data is routed into the dedicated
OpenShift AI Prometheus instance (for metrics) and Tempo (for distributed tracing).
Platform Engineers visualize this aggregated data in centralized Grafana dashboards.
To export metrics to an existing enterprise observability tool (such as Splunk,
Datadog, or an external Prometheus), Platform Engineers configure the
`spec.monitoring.metrics.exporters` field in the **DataScienceClusterInitialization
(DSCI)** custom resource, which automatically configures the OpenTelemetry Collector
to forward metrics to external destinations.

[discrete#chapter8:section1:::_model_performance_telemetry_ttft_itl_and_saturation]
==== Model Performance Telemetry: TTFT, ITL, and Saturation
Platform Engineers must monitor specific LLM metrics to gauge user experience and
trigger autoscaling, rather than relying on generic CPU monitoring:

* **Time-To-First-Token (TTFT):** Measures the latency from when a user sends a prompt
to when the first word appears. High TTFT usually indicates an overloaded queue.
* **Time-Per-Output-Token (TPOT / ITL):** Measures the inter-token latency (how fast
the text streams). If this slows down, the GPU is likely struggling with compute or
memory bandwidth.
* **KV Cache Saturation:** LLMs store attention keys/values in the GPU's KV Cache
(`vllm:gpu_cache_usage_perc`). If the KV cache hits 100%, the model cannot accept new
requests. The system queues them (`vllm:num_requests_waiting`). The new 3.4 Workload
Variant Autoscaler (WVA) watches these exact saturation metrics to intelligently
trigger horizontal pod autoscaling before the queue gets too long.

[discrete#chapter8:section1:::_usage_tracking_showback_frameworks]
```

```
==== Usage Tracking & Showback Frameworks
Spiraling cloud costs are a major risk in generative AI. OpenShift AI 3.4 solves this
with the Models-as-a-Service (MaaS) Observability Dashboard.

By enabling telemetry on the MaaS `Tenant` custom resource — specifically setting
`spec.telemetry.enabled: true` and `captureModelUsage: true` on the `default-tenant`
in the `models-as-a-service` namespace — the MaaS API gateway (backed by Kuadrant and
Limitador) actively tracks every single inference request. The dashboard aggregates
this data, allowing the Platform Engineer to see exactly how many tokens were consumed
by specific users, groups, or Subscriptions.

This data can be exported as a CSV file to provide Finance with an enterprise-grade
"Showback" report, attributing specific LLM API costs directly to the business units
consuming them.

For monitoring distributed workload resource consumption (CPU and memory usage across
distributed training and inference jobs), navigate to **Observe & monitor -> Workload
metrics** in the OpenShift AI dashboard. This view provides project-level aggregation
of distributed workload resource usage across all tenants.

[discrete#chapter8:section1:::_from_showback_to_business_value]
==== From Showback to Business Value
Showback is the starting point, not the end state. Organizations progressing along the
AI platform maturity curve should plan for:

* **Showback to Chargeback transition:** Moving from "visibility into costs" to
"actual cost allocation to business units" using the CSV export data as input to
FinOps tooling.
* **Platform success metrics:** Track developer adoption rate (active API keys / total
developers), time-to-first-inference (from onboarding to first successful API call),
and model deployment velocity (models promoted from registry to production per
sprint).

These metrics demonstrate platform value to leadership and justify continued
investment in the self-service AI infrastructure. _(See supplemental rhoai3-deploy and
rhoai3-validate content for ROI calculation formulas and platform maturity models.)_

[discrete#chapter8:section1:::_day_2_diagnostics_the_anti_pattern]
==== Day-2 Diagnostics: The Anti-Pattern
*Can you spot the architectural flaw?*

**The Scenario: The CPU Scaling Trap**
A Platform Engineer deploys a massive 70B parameter LLM for the customer support team.
To handle traffic spikes, they configure a standard Kubernetes Horizontal Pod
Autoscaler (HPA) to scale the model replicas whenever CPU utilization exceeds 80%.
They also write a custom sidecar Python script to parse the raw text logs of the pod
to count tokens for financial showback.
During peak hours, user requests start timing out with 500 errors. The engineer checks
the cluster and sees that the HPA never triggered a scale-up because the CPU was only
at 30%.
```

```
**Why this is an Anti-Pattern:**
* **Scaling Anti-Pattern:** LLM inference is fundamentally GPU and Memory-bound, not
CPU-bound. The GPU's KV Cache filled up to 100%, causing the queue to overflow and
drop requests, all while the CPU remained mostly idle. Standard CPU-based HPA is
useless for LLMs.
* **Showback Anti-Pattern:** Parsing raw pod logs to count tokens is fragile, misses
cached requests, and puts massive unnecessary load on the logging infrastructure.

**The Solution:**
In OpenShift AI 3.4, the Platform Engineer must use the native AI telemetry
architecture.
* **For Scaling:** They deploy the model using the **Workload Variant Autoscaler
(WVA)**. The WVA scales based on actual LLM saturation signals—specifically the
`vllm:kv_cache_usage_perc` and `vllm:num_requests_waiting` metrics.
* **For Showback:** They enable **MaaS Telemetry**. The MaaS API Gateway natively
tracks exact token counts and feeds the data directly into the MaaS Observability
Dashboard for a clean, CSV-exportable showback report.

<<<
*Ready to build it? Proceed to the Interactive Labs.*

:!navtitle:
:!description:

:docname: section2
:page-module: chapter8
:page-relative-src-path: section2.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
:navtitle: Labs: Telemetry & Showback
:description: Hands-on walk-throughs for enabling User Workload Monitoring, MaaS
Showback tracking, and scraping custom vLLM metrics.
[#chapter8:section2]
=== Labs: Telemetry &amp;amp; Showback

In these interactive labs, you will configure the essential telemetry backbone of
OpenShift AI, enabling financial accountability and performance tracing.

[discrete#chapter8:section2:::_lab_1_enabling_the_centralized_observability_stack]
==== Lab 1: Enabling the Centralized Observability Stack
**Goal:** Deploy the OpenTelemetry, Prometheus, and Tempo stack to collect metrics
across the cluster.

.Arcade: Enable Core Observability
====
// Antora iframe embed placeholder for Arcade 1
// ++++
// <iframe src="URL_TO_ARCADE_1" width="100%" height="600px"></iframe>
```

```
// ++++

**Guided Steps:**
1. Log in to the OpenShift web console as a cluster administrator.
2. Navigate to **Ecosystem -> Installed Operators** and click the **Red Hat OpenShift
AI Operator**.
3. Click the **DSCInitialization** tab and click the `default-dsci` instance.
4. Select the **YAML** tab.
5. In the `spec.monitoring` section, set the `managementState` to `Managed`. Ensure
you define the metrics storage size (e.g., `size: 5Gi`). Click **Save**.
6. *To make the dashboard visible:* Navigate to **Home -> API Explorer**, search for
`OdhDashboardConfig`, click the `odh-dashboard-config` instance, and edit the YAML to
set `spec.dashboardConfig.observabilityDashboard: true`.
====

[discrete#chapter8:section2:::_lab_2_enabling_maas_telemetry_for_financial_showback]
==== Lab 2: Enabling MaaS Telemetry for Financial Showback
**Goal:** Instruct the MaaS Gateway to capture token usage data for cost attribution.

.Arcade: Configure Showback Export
====
// Antora iframe embed placeholder for Arcade 2
// ++++
// <iframe src="URL_TO_ARCADE_2" width="100%" height="600px"></iframe>
// ++++

**Guided Steps:**
1. In the OpenShift console, navigate to **Administration ->
CustomResourceDefinitions**.
2. Search for `Tenant` and click the resource name.
3. Click the **Instances** tab and click `default-tenant`. Select the **YAML** tab.
4. In the `spec.telemetry` section, set `enabled: true`.
5. Under metrics, set `captureModelUsage: true` and `captureOrganization: true`. Click
**Save**.
6. Open the OpenShift AI Dashboard, navigate to **Observe & monitor -> Dashboard**,
click the **Usage** tab, and click **Export as CSV** to download the billing showback
report for Finance.
====

[discrete#chapter8:section2:::_lab_3_scraping_custom_user_workload_metrics]
==== Lab 3: Scraping Custom User Workload Metrics
**Goal:** Ensure Prometheus is actively scraping a data science team's deployed model
so you can track KV Cache saturation.

.Arcade: Enable Custom Scraping
====
// Antora iframe embed placeholder for Arcade 3
// ++++
// <iframe src="URL_TO_ARCADE_3" width="100%" height="600px"></iframe>
// ++++
```

```
**Guided Steps:**
1. Log in to the OpenShift web console and navigate to **Workloads -> Deployments**.
2. Select the namespace of the data science project where the model is deployed.
3. Click the deployment of the model server (e.g., `vllm-server`), and click the
**YAML** tab.
4. In the `spec.template.metadata.labels` section, add the following exact label:
`monitoring.opendatahub.io/scrape: 'true'`. Click **Save**.
5. Navigate to **Observe -> Metrics** in the Developer perspective. Enter
`vllm:gpu_cache_usage_perc` into the custom query box and click **Run queries** to
view the live KV cache saturation of the newly labeled pod.
====

**Course Complete!** You have constructed the AI Engine Room from the ground
up—architecting operator fabrics, scaling high-density inference pipelines, governing
accelerator hardware, standardizing RAG prototyping, and establishing strict financial
showback controls.

:!navtitle:
:!description:

:docname: appendix
:page-module: appendix
:page-relative-src-path: appendix.adoc
:page-origin-url: https://github.com/RedHatQuickCourses/rhoai34-practical
:page-origin-start-path:
:page-origin-refname: main
:page-origin-reftype: branch
:page-origin-refhash: (worktree)
[#appendix:appendix]
== Appendix

[discrete#appendix:appendix:::_supplemental_deep_dive_resources]
=== Supplemental Deep-Dive Resources
The chapter runbooks reference companion repositories that provide advanced deep-dive
content for specific topics. These resources extend your foundational knowledge into
specialized domains:

* **rhoai3-mass2** — Advanced MaaS multi-tenant governance patterns, preemption
policies, cross-subscription fairness, and enterprise-scale rollout playbooks.
* **rhoai3-llmd** — llm-d distributed inference tuning: EPP optimization,
prefill/decode routing, and multi-node NCCL configuration.
* **rhoai3-validate** — TrustyAI Guardrails Orchestrator implementation, garak red-
teaming deep-dives, compound AI systems, and ROI calculation formulas.
* **rhoai3-gpu-aas** — MIG partition strategies (1g.5gb, 2g.10gb, 3g.20gb tables) and
detailed GPU sharing configurations.
* **rhoai3-hwprofiles** — Advanced Kueue preemption policies and fair-share scheduling
for multi-team GPU governance.
* **rhoai3-storage** — Storage performance anti-patterns, OCI caching strategies,
egress cost optimization, and large model pull networking.
* **rhoai3-deploy** — Cost modeling, platform maturity models, business case formulas,
and vLLM performance tuning guides.
```

```
* **rhoai3-journey** — End-to-end learning path covering Phases 0-4 (Operators through
Compound AI Systems and autonomous agent patterns).

[discrete#appendix:appendix:::_resources]
=== Resources

link:{attachmentsdir}/{page-component-name}.pdf[Download Course PDF]
```